



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Function and type definition	
Topic Code	-	
Unit/Module No. & Name	1	Standard Constructs
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14-15



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. A function is a fundamental concept in discrete mathematics and computer science.
2. It describes how elements of one set are linked to elements of another set using a clear rule.
3. The purpose of this paper is to explain functions in simple English while maintaining an academic undergraduate tone.
4. It introduces common types of functions, including one-to-one, onto, and bijective functions.
5. These function types are widely used in proofs, data structures, and programming.
6. Understanding these ideas prepares students for advanced topics such as relations, proofs, and algorithms.
7. Clear mapping rules help prevent confusion and logical errors in later studies.

1.0 INTRODUCTION

1.1 Why Functions Matter

In many subjects, students learn about functions at an early stage. A function can be understood as a simple machine that takes an input and produces an output. For example, a computer program can take a number, square it, and return the result. In databases, a function can link a student ID to the student's record. In logic, functions help describe relationships between objects clearly and accurately. Because of these uses, functions support later topics such as relations, graphs, algorithms, and counting.

Functions also help students talk clearly about ideas like “same” and “different.” If two inputs give the same output, we can ask why this happens and what it means. If every possible output is reached, we can connect this idea to completeness and coverage. These ideas are important in topics like hashing, encryption, and inverse operations.

1.2 Functions As Models

In daily life, many rules work like functions. For example, a rule can assign each person a national ID number. Another rule can assign each date a day of the week. In science, a rule may

assign a temperature value to each time point. These examples show that a function is not only a mathematical idea, but also a way to model real processes where one input leads to one output.

However, we must be careful. A real-life rule may fail to be a function if one input can give more than one output. For example, a student may attend different classes at different times. If a rule does not give one clear output, it is not a function unless we clearly define or refine the input or the output.

1.3 What This Paper Covers

This paper first explains what a function is and gives its definition, including important terms such as domain, codomain, and range. Next, it explains different types of functions, such as injective, surjective, and bijective functions. Finally, it gives examples and short case studies that connect these ideas to real-life situations and computing tasks. The aim is to help students understand the ideas clearly, using simple language while keeping the concepts accurate and meaningful.



2.0 MEANING AND DEFINITION

2.1 Meaning Of A Function

A function is a special kind of relation. It is a rule that links each input to exactly one output. The word “exactly” matters. It means the rule cannot give two different outputs for the same input. It also means the rule must give some output for every input that is allowed.

To picture a function, imagine a box. You put one item into the box, and the box gives one item out. If you put the same input into the box again, the box should give the same output, because the rule is fixed. This idea of a fixed rule is important for trust and for reasoning in math and computing.

2.2 Standard Definition Using Sets

Let A and B be sets. A function f from A to B , written $f: A \rightarrow B$, is a rule that assigns to each element a in A exactly one element b in B . Here, A is the set of inputs, and B is the set of possible outputs. The function name, such as f , helps us talk about the rule.

This definition includes two key limits. First, every element in A must have an image in B . Second, each element in A must map to only one element in B . The definition does not require that every element in B must be used. That extra condition belongs to a special type called “onto,” which is explained later.

2.3 Domain, Codomain, And Range

The domain of a function is the set A . It contains all inputs that the rule accepts. The codomain is the set B . It contains all outputs that the rule is allowed to produce. The range, also called the image, is the set of outputs that the function actually produces when it runs over the entire domain.

These three words sound similar, so a careful example helps. Suppose $A = \{1, 2, 3\}$ and $B = \{2, 4, 6, 8\}$. Define $f(x) = 2x$. Then $f(1) = 2$, $f(2) = 4$, and $f(3) = 6$. The domain is $\{1, 2, 3\}$. The codomain is $\{2, 4, 6, 8\}$. The range is $\{2, 4, 6\}$. Notice that 8 is in the codomain but not in the range.

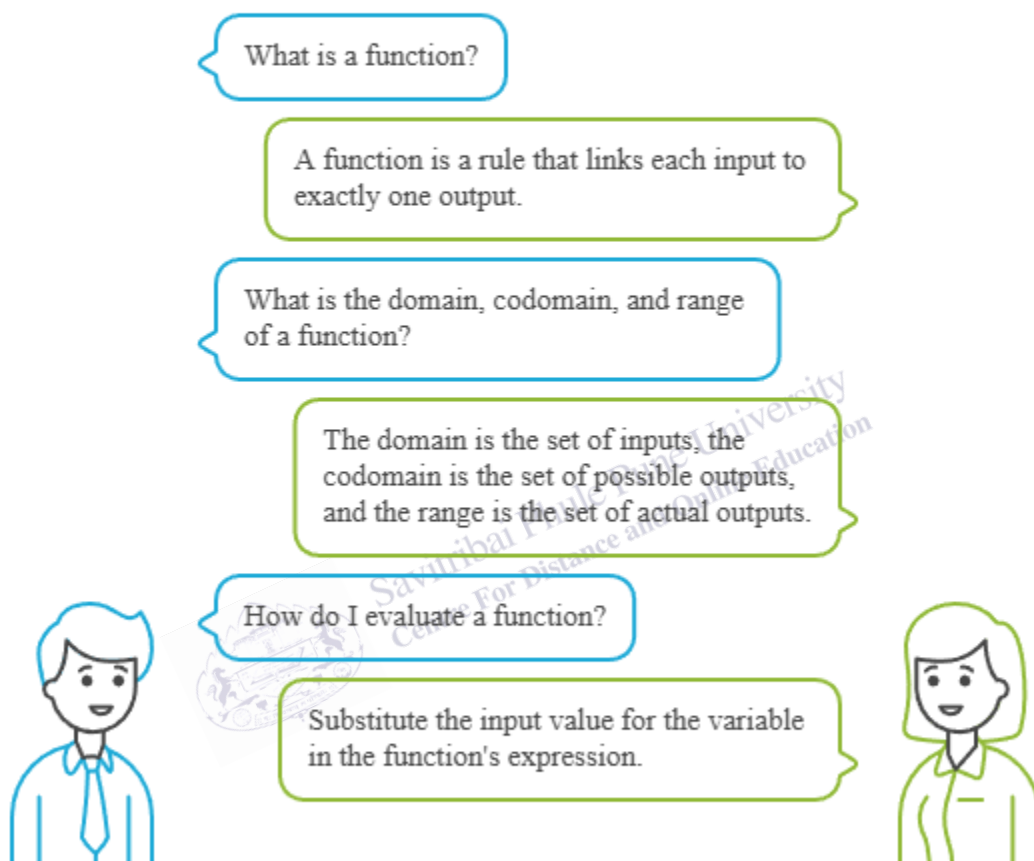
2.4 Function Notation And Evaluation

A function is often written as $f(x)$. The symbol x stands for an input. The expression $f(x)$ stands for the output produced by applying the rule to x . For example, if $f(x) = x + 5$, then $f(2) = 7$. Here, the input is 2, and the output is 7.

In computing, we can think of $f(x)$ as a function call. The value x is passed as an argument, and the returned value is the result. This connection helps students move between math notation and program notation. It also shows why functions are central in software design.

Fig. 1

Understanding Functions



The image introduces functions by defining input–output rules, explaining domain, codomain, and range, and showing how to evaluate a function. It uses simple dialogue-style visuals to make abstract mathematical ideas clear, student-friendly, and suitable for undergraduate learning.

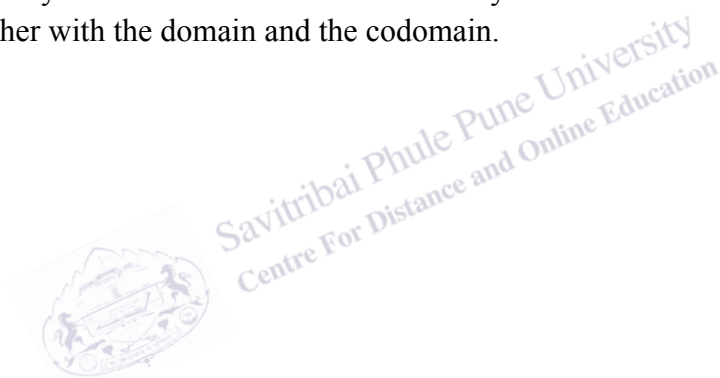
A function can also be described in more than one form. We may list ordered pairs, such as (1, 2), (2, 4), and (3, 6). We may draw an arrow diagram that shows arrows from elements of the domain to elements of the codomain. We may also draw a graph on coordinate axes when the inputs and outputs are numbers. Each form shows the same rule, but each form supports a different skill. Lists support exact checking, arrow diagrams support set thinking, and graphs

support visual reasoning. In all forms, the key test is the same: each input has one arrow, one partner, or one point above it. The word preimage is also useful. If b is an output, then the preimage of b is the set of inputs that map to b . For an injective function, each output has at most one preimage element.

2.5 When A Rule Is Not A Function

Some rules look like functions but fail the “exactly one output” rule. For example, consider the rule that assigns to each real number x the set of numbers y such that $y^2 = x$. If $x = 4$, then y can be 2 or -2 . Because one input has two outputs, this rule is not a function unless we choose one branch, such as the nonnegative square root. Another common issue is leaving an input without any output. For instance, the rule $g(x) = 1/x$ is not defined at $x = 0$ if the domain includes 0. To make it a function, we must restrict the domain to nonzero real numbers, or we must change the rule so it gives a value at 0. In class, this step is called “defining the domain.” It is an academic way of saying, “Tell us which inputs are allowed, so the rule always works.”

These points show why the domain must be stated clearly. A function is not only the formula. It is the formula together with the domain and the codomain.



3.0 NATURE OR TYPE (IF APPLICABLE)

3.1 Why Types Of Functions Are Studied

Once students know what a function is, they often ask stronger questions. Does each input lead to a different output, or can two inputs collide? Does the function reach every possible output in the codomain, or does it miss some? Can we reverse the process and find an input from an output? These questions lead to important types of functions.

In computing, these questions are not only theoretical. Collisions matter in hashing. Coverage matters in testing. Reversibility matters in encryption and decoding. Therefore, the types of functions described below are practical ideas with clear meaning.

3.2 One-To-One (Injective) Functions

3.2.1 Definition

A function is one-to-one, also called injective, if different inputs always give different outputs. In symbols, f is injective if whenever $f(a_1) = f(a_2)$, we must have $a_1 = a_2$. This statement may feel backward at first, but it is a precise way to capture “no two inputs share one output.”

3.2.2 Meaning In Simple Words

Injective means the function never merges two separate inputs into the same result. Each output in the range points back to at most one input. This feature makes injective functions useful for identification tasks. If a code is injective, then each code refers to one item only.

3.2.3 Quick Example

Let $A = \{1, 2, 3\}$ and define $f(x) = x + 10$. Then the outputs are $\{11, 12, 13\}$. No two inputs give the same output, so the function is injective. In contrast, the function $h(x) = x^2$ from integers to integers is not injective because $h(2) = 4$ and $h(-2) = 4$.

3.3 Onto (Surjective) Functions

3.3.1 Definition

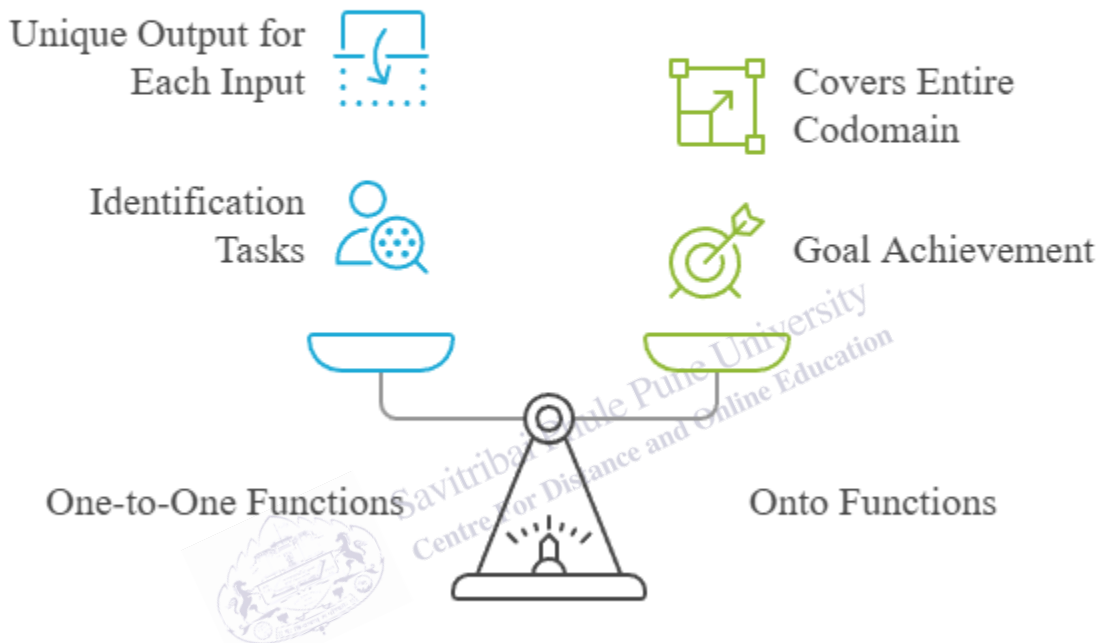
A function $f: A \rightarrow B$ is onto, also called surjective, if every element of B is the image of at least one element of A . In other words, for every b in B , there exists some a in A such that $f(a) = b$. When this happens, the range of f equals the codomain B .

3.3.2 Meaning In Simple Words

Surjective means the function covers the whole codomain. Nothing in the codomain is left unused. This idea is important when B represents goals, options, or states that must all be reachable. If some outputs can never occur, then the model may be incomplete.

Fig. 2

Balancing Function Types for Computing



The image compares one-to-one and onto functions using a balance scale. It shows how injective functions provide unique outputs for identification tasks, while surjective functions cover the entire codomain to achieve complete goals in computing contexts.

3.3.3 Quick Example

Let $A = \{1, 2, 3, 4\}$ and $B = \{0, 1\}$. Define $f(x) = x \bmod 2$, which returns the remainder when x is divided by 2. Then $f(1) = 1$, $f(2) = 0$, $f(3) = 1$, and $f(4) = 0$. Both 0 and 1 appear, so the function is surjective onto B . However, if we use $A = \{1, 2\}$ and $B = \{0, 1, 2\}$, no function from A to B can be surjective because A has too few inputs to hit all outputs.

3.4 Bijective Functions

3.4.1 Definition

A function is bijective if it is both injective and surjective. That means it gives different outputs for different inputs, and it also reaches every output in the codomain.

3.4.2 Meaning In Simple Words

Bijective means the function makes a perfect pairing between the domain and the codomain. Each input matches one output, and each output matches one input. Because of this pairing, a bijective function can be reversed. The reverse rule is called the inverse function.

3.4.3 Quick Example

Let $A = \{1, 2, 3\}$ and $B = \{4, 5, 6\}$. Define $f(1) = 4$, $f(2) = 5$, and $f(3) = 6$. This function is injective because outputs are distinct, and it is surjective because every element of B is used. So it is bijective. The inverse function f^{-1} maps 4 back to 1, 5 back to 2, and 6 back to 3.

3.5 Inverse Functions And Why They Need Bijection

An inverse function undoes the work of a function. If f maps A to B , and f is bijective, then there exists a function f^{-1} that maps B back to A . The inverse follows the rule $f^{-1}(b) = a$ exactly when $f(a) = b$. This statement says that the inverse swaps the roles of input and output.

The need for bijection is clear if we test the two failures. If f is not injective, then two different inputs share one output, so the inverse would not know which input to return. If f is not surjective, then some outputs in B are never hit, so the inverse would have no answer for those outputs. Therefore, bijection is the condition that makes an inverse a true function.

4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Assigning Student Roll Numbers

Suppose a college assigns each student a roll number, and no two students share the same roll number. Then the mapping from students to roll numbers is injective. Different students give different roll numbers. This rule supports fair record keeping because one roll number points to one student.

Is it surjective? That depends on the codomain. If the codomain is “all roll numbers that exist in the college database,” then it can be surjective if every stored roll number belongs to some student. If the codomain is “all possible 10-digit numbers,” then it is not surjective because most numbers are unused. This example shows why codomain choice affects the type.

4.2 Example: Aadhaar Number To Person

A national ID system aims to link each ID number to exactly one person. The mapping from an Aadhaar number to a person can be seen as a function because each valid number should identify one person. In this direction, it is injective if different Aadhaar numbers belong to different people.

In the other direction, mapping people to Aadhaar numbers is also intended to be a function if each person has exactly one number. If the system is well maintained, this mapping can be bijective between the set of issued Aadhaar numbers and the set of registered people. In practice, administrators work to prevent duplicates, because duplicates would break injectivity and harm trust.

4.3 Example: Squaring Integers

Consider the function $f(x) = x^2$ from integers to integers. This function is not injective because $f(2) = 4$ and $f(-2) = 4$. Two different inputs collide into one output. It is also not surjective onto the integers because negative integers are never produced by squaring. The range contains only nonnegative integers, so many codomain values are missed.

This example is important in algebra and in proofs because it shows that a simple formula can fail to be injective or surjective depending on the chosen domain and codomain. If we restrict the domain to nonnegative integers and restrict the codomain to nonnegative integers, the function becomes injective, but it still is not surjective unless we also limit the codomain to perfect squares.

4.4 Example: Modulo As A Many-To-One Map

The modulo function maps an integer to its remainder after division. For example, $x \bmod 5$ always returns one of $\{0, 1, 2, 3, 4\}$. Many different integers share the same remainder, so modulo is usually not injective when the domain is all integers. However, it can be surjective onto the set of remainders because every remainder can occur.

Modulo functions are used in computing for cycling through positions, building hash tables, and grouping data into buckets. The fact that it is not injective is not always a problem. In hashing, collisions are expected, and the design goal is to manage them well.

4.5 Case Study: Hashing In A Dictionary Data Structure

A dictionary, also called a map, stores key–value pairs. To find a value quickly, the system may compute a hash of the key. The hash function sends many possible keys into a smaller set of bucket numbers. Because there are more keys than buckets, collisions must happen, so the hash function cannot be injective over all possible keys. Still, designers want collisions to be rare for typical data, because fewer collisions mean faster search.

This case study connects math to practice. The idea of injective is like a gold standard for uniqueness, but it may be impossible when compressing a large set into a small one. In such cases, we use a function that is “close enough” and then add collision-handling methods, such as chaining or open addressing. The academic point is that type properties guide design choices and explain limits.

4.6 Case Study: Encoding And Decoding As Inverse Functions

Many systems encode information into another form and later decode it back. For decoding to work as a function, the encoding must be injective on the set of messages we allow. If two different messages had the same code, the decoder could not know which message was meant. When the code space and message space match well, we may even aim for a bijection, because then encoding and decoding are exact inverses.

A simple example is mapping letters A, B, C, ... to numbers 1, 2, 3, If the map is one-to-one and uses all allowed numbers, then it is bijective between the chosen sets. In real systems, codes may include extra symbols for error checking, which means the codomain is larger than the range. Then the function is injective but not surjective, and decoding still works for valid codes.

4.7 Case Study: Testing Coverage With Surjective Thinking

Software testers often want to cover all important output cases. If a function in a program is meant to produce all states in a list, then the function should be surjective onto that list. If some

states never occur, that might mean the code has a bug, or the state list is too broad. Surjective thinking helps teams ask, “Do we reach every planned case?”

For example, suppose a grading program should output one of $\{A, B, C, D, F\}$. If a bug makes it impossible to output D, then the function from scores to grades is not surjective onto the intended codomain. A careful test plan can reveal the missing output and guide the fix.

1. A function assigns exactly one output to each allowed input.
2. The domain is the input set, the codomain is the allowed output set, and the range is the actual output set.
3. A rule fails to be a function if an input has two outputs or if some allowed input has no output.
4. Injective means no two different inputs share one output.
5. Surjective means every element of the codomain is reached.
6. Bijective means both injective and surjective, so an inverse function exists.
7. The same formula can change type when we change the domain or codomain.
8. Real systems like IDs, hashing, and encoding show why function types matter in practice.
9. Surjective thinking supports coverage goals, and injective thinking supports uniqueness goals.
10. Clear set choices and clear definitions make mathematical claims testable.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning	Touch Vocabulary In Use
Function	A rule from inputs to outputs	The function maps each input to one output.
Domain	Set of inputs	The domain tells which inputs are allowed.
Codomain	Set of allowed outputs	The codomain lists possible outputs in advance.
Range (Image)	Outputs actually produced	The range may be smaller than the codomain.
Map	To connect by a rule	The rule maps a to b.
Injective	One-to-one	Injective means no collisions in outputs.
Surjective	Onto	Surjective means every target is hit.
Bijective	One-to-one and onto	Bijective creates a perfect pairing.
Inverse	Undoing rule	The inverse sends outputs back to inputs.

Collision	Two inputs share one output	Hashing manages collisions carefully.
Modulo	Remainder after division	Modulo groups inputs into repeating classes.
Proof	Logical argument	Proofs use function types to show structure.

